

Lab Exercise 5- Implementation of API Security using Multi-Factor Authentication (MFA) (OTP)

1. Objective

The objective of this lab exercise is to implement **Multi-Factor Authentication (MFA)** in a REST API using Spring Boot. MFA enhances security by requiring users to verify their identity using two factors:

- Password (something the user knows)
- One-Time Password (OTP) sent to the user (something the user receives)

This mechanism protects APIs from unauthorized access even if passwords are compromised.

2. Learning Outcomes

After completing this lab exercise, students will be able to:

- Understand the concept of Multi-Factor Authentication (MFA)
 - Implement OTP-based authentication for APIs
 - Secure REST APIs using two-step verification
 - Test MFA workflows using Postman
 - Understand practical API security mechanisms used in industry
-

3. Software Requirements

Software	Purpose
Java JDK 17	Programming language
VS Code	Development environment
Maven	Dependency management
Postman	API testing
Spring Boot	Backend framework

4. Project Creation

Step 1

Open Spring Initializr.

Project configuration

Field	Value
Group	com.example
Artifact	mfaapi
Packaging	Jar
Java Version	17

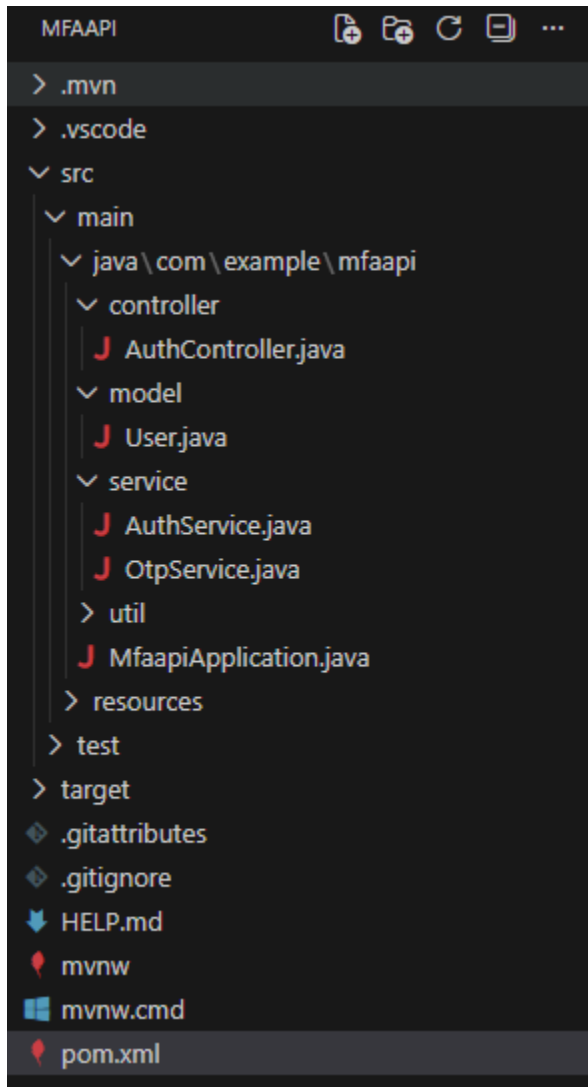
Dependencies

- Spring Web
- Spring Boot DevTools
- Lombok (optional)

Download the project and open it in **VS Code**.

5. Project Structure

Create the following folder structure.



6. Create User Model

File

model/User.java

Code

```
package com.example.mfaapi.model;

public class User {

    private String username;
    private String password;
    private String otp;

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }
}
```

```
public void setPassword(String password) {  
    this.password = password;  
}  
  
public String getOtp() {  
    return otp;  
}  
  
public void setOtp(String otp) {  
    this.otp = otp;  
}  
}
```

Explanation

This class represents the **user authentication request** containing:

- username
- password
- OTP

7. Create OTP Service

File

service/OtpService.java

Code

```
package com.example.mfaapi.service;

import java.util.Random;
import java.util.concurrent.ConcurrentHashMap;

import org.springframework.stereotype.Service;

@Service
public class OtpService {

    private ConcurrentHashMap<String, String> otpStorage =
        new ConcurrentHashMap<>();

    public String generateOtp(String username){

        Random random = new Random();

        int otp = 100000 + random.nextInt(900000);

        String otpValue = String.valueOf(otp);

        otpStorage.put(username, otpValue);

        return otpValue;
    }

    public boolean validateOtp(String username, String otp){

        String storedOtp = otpStorage.get(username);

        if(storedOtp != null && storedOtp.equals(otp)){
```

```
        otpStorage.remove(username);  
        return true;  
    }  
  
    return false;  
}  
  
}
```

Explanation

This service performs:

- OTP generation
- Temporary OTP storage
- OTP validation

8. Create Authentication Service

File

service/AuthService.java

Code

```
package com.example.mfaapi.service;  
  
import org.springframework.stereotype.Service;  
  
@Service  
public class AuthService {  
  
    public boolean validateUser(String username, String password){
```

```
    if(username.equals("admin") &&
        password.equals("password")){

        return true;
    }

    return false;
}
```

Explanation

This service simulates **user authentication** using username and password.

9. Create MFA Controller

File

controller/AuthController.java

Code

```
package com.example.mfaapi.controller;

import org.springframework.web.bind.annotation.*;

import com.example.mfaapi.model.User;
import com.example.mfaapi.service.AuthService;
import com.example.mfaapi.service.OtpService;

@RestController
```



```
@RequestMapping("/auth")
public class AuthController {

    private final AuthService authService;
    private final OtpService otpService;

    public AuthController(AuthService authService,
                          OtpService otpService){

        this.authService = authService;
        this.otpService = otpService;
    }

    @PostMapping("/login")
    public String login(@RequestBody User user){

        boolean validUser =
            authService.validateUser(
                user.getUsername(),
                user.getPassword());

        if(!validUser){
            return "Invalid credentials";
        }

        String otp =
            otpService.generateOtp(user.getUsername());

        return "OTP Generated: " + otp;
    }

    @PostMapping("/verify-otp")
```

```
public String verifyOtp(@RequestBody User user){

    boolean validOtp =
        otpService.validateOtp(
            user.getUsername(),
            user.getOtp());

    if(validOtp){
        return "MFA Authentication Successful";
    }

    return "Invalid OTP";
}
}
```

Explanation

This controller performs **two authentication steps**.

Step 1

Username and password verification.

Step 2

OTP verification.

10. Run the Application

Open terminal in VS Code.

Run the application.

```
mvn spring-boot:run
```

Server runs at

<http://localhost:8080>

11. API Testing Using Postman

Test 1 — User Login

POST

`http://localhost:8080/auth/login`

Body

```
{  
  "username": "admin",  
  "password": "password"  
}
```

Response

OTP Generated: 458932

Test 2 — Verify OTP

POST

`http://localhost:8080/auth/verify-otp`

Body

```
{  
  "username": "admin",  
  "otp": "458932"
```

```
}
```

Response

```
MFA Authentication Successful
```

Test 3 — Invalid Password

Request

POST

```
{  
  "username": "admin",  
  "password": "wrongpass"  
}
```

Response

```
Invalid credentials
```

Test 4 — Invalid OTP

Request

```
{  
  "username": "admin",  
  "otp": "111111"  
}
```

Response

Invalid OTP

12.Security Workflow

The MFA authentication flow works as follows.

Step 1

User sends username and password.

Step 2

Server verifies credentials.

Step 3

Server generates OTP.

Step 4

User submits OTP.

Step 5

Server verifies OTP.

Step 6

User gets authenticated.

13.Use Case Example

Banking API Authentication

Step Process

- 1 User enters username and password
- 2 Bank sends OTP to mobile/email

Step Process

- 3 User enters OTP
- 4 System verifies OTP
- 5 User gains secure access

Even if a password is compromised, attackers cannot access the account without the OTP.